

Table Relationships

Product Shop

รหัสนักศึกษา: 673380062-2 Section: 1

ชื่อโปรเจกต์: lab8-673380062-2-sec1

ระบบจัดการสินค้าโดยใช้ Spring Boot 3.3.0, JPA, Thymeleaf และ PostgreSQL พร้อมข้อมูลเสริมแบบ 1:1 และรีวิวแบบ 1:N

ส่วนที่ 1: หลักการออกแบบ - SOLID

S - Single Responsibility: Product เก็บข้อมูลหลักของสินค้า ส่วน ProductDetail เก็บรายละเอียดเพิ่มเติม และ Review เก็บความคิดเห็นแต่ละครั้ง Controller ดูแล HTTP, Service ดูแลตรรกกรรมและกฎการทำงาน ส่วน Repository ติดต่อฐานข้อมูล

O - Open/Closed: DiscountContext รับรายการ DiscountStrategy ผ่าน constructor จึงเพิ่มคลาสส่วนลดชนิดใหม่โดยไม่แก้การเลือก strategy ใน context อย่างไรก็ตามต้องเพิ่มตัวเลือกในหน้าฟอร์มด้วย ส่วนการเพิ่มรีวิวเป็นการเพิ่มแถว ไม่ต้องเปลี่ยนโครงสร้างตารางสินค้า

L - Liskov Substitution: NoDiscountStrategy, MemberDiscountStrategy และ SeasonalSaleStrategy ใช้แทน DiscountStrategy ได้โดยรับราคาและคืนราคาสุทธิตามสัญญาเดียวกัน ไม่ได้หมายความว่า Repository ของคนละ Entity ใช้แทนกันได้

I - Interface Segregation: DiscountStrategy มีเพียง type() และ apply() ส่วน Repository แยกตาม Entity ทำให้แต่ละส่วนเรียกใช้ interface ที่เกี่ยวข้องกับงานของตน

D - Dependency Inversion: ProductService รับ ProductRepository ซึ่งเป็น interface และ DiscountContext รับ List<DiscountStrategy> จาก Spring แทนการสร้าง implementation ด้วย new ภายในคลาส

โค้ดและภาพหน้าจอในรายงานมาจากโปรเจกต์ที่สร้างและทดสอบจริง วันที่ 8 กันยายน 2026

1:1, 1:N และ Strategy Pattern

One-to-One: Product กับ ProductDetail

สินค้าแต่ละรายการมีข้อมูลเสริมหนึ่งรายการ เช่น คำอธิบาย หน้าหนัก และการรับประกัน Product เป็น owning side โดยเก็บ detail_id ซึ่งอ้างอิง product_details.id และมี UNIQUE ป้องกันสินค้าหลายรายการอ้างอิงรายละเอียดเดียวกัน เหมาะกับข้อมูลเสริมที่มีชุดเดียวต่อสินค้าหนึ่งรายการ

One-to-Many: Product กับ Review

สินค้าแต่ละรายการมีรีวิวได้ตั้งแต่ศูนย์ถึงหลายรายการ ส่วนรีวิวแต่ละรายการเป็นของสินค้าหนึ่งรายการ Review เป็น owning side และเก็บ product_id ส่วน Product.reviews ใช้ mappedBy="product" จึงเป็น inverse side โดยไม่มีตารางกลาง

```
products.detail_id [UNIQUE, FK] ---> product_details.id [PK]
reviews.product_id [FK] ---> products.id [PK]
Product 1 <-----> 1 ProductDetail
Product 1 <-----> 0..N Review
```

การคำนวณส่วนลด

NONE: ราคาเดิม, MEMBER: ลด 10%, SEASONAL: ลด 20% แต่ละ strategy ใช้ BigDecimal คำนวณและบัต HALF_UP เป็นสองตำแหน่ง ก่อนคืน Double ตามชนิดข้อมูลที่โจทย์กำหนด ค่า discountedPrice ใช้ @Transient จึงไม่ถูกบันทึกลงฐานข้อมูล

```
NONE: 40,000 x 1.00 = 40,000.00
MEMBER: 40,000 x 0.90 = 36,000.00
SEASONAL: 39,000 x 0.80 = 31,200.00
```

Execution Flow: HTTP Request ไปถึงฐานข้อมูล

1. ผู้ใช้ส่ง POST /products/save จากฟอร์ม Thymeleaf
2. Controller bind ค่าลง ProductForm และตรวจ @Valid พร้อม BindingResult
3. ProductService.create() สร้าง Product, ProductDetail และรีวิวที่มีข้อมูล โดยดึงความสัมพันธ์ทั้งสองฝั่ง
4. ภายใน @Transactional เรียก ProductRepository.save() จากนั้น Hibernate บันทึกแถวและ Foreign Key ใน PostgreSQL ด้วย cascade
5. เมื่อสำเร็จ redirect ไป GET /products ซึ่งโหลดข้อมูลพร้อมความสัมพันธ์ คำนวณส่วนลด และ render ตารางสินค้า

Product.java

```
package com.example.demo.model;
import jakarta.persistence.*;
import java.util.*;
@Entity
@Table(name = "products")
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String name;
    @Column(nullable = false)
    private String category;
    @Column(nullable = false)
    private String brand;
    @Column(nullable = false)
    private Integer stock;
    @Column(nullable = false)
    private Double price;
    @Column(nullable = false)
    private String discountType = "NONE";
    @OneToOne(cascade = CascadeType.ALL, orphanRemoval = true)
    @JoinColumn(name = "detail_id", unique = true, nullable = false)
    private ProductDetail detail;
    @OneToMany(mappedBy = "product", cascade = CascadeType.ALL, orphanRemoval = true)
    @OrderBy("id ASC")
    private List<Review> reviews = new ArrayList<>();
    @Transient
    private Double discountedPrice;
}
```

@Entity / @Table: แมปคลาสไปยังตาราง products; **@Id / @GeneratedValue:** ใช้ primary key แบบ identity ที่ฐานข้อมูลสร้างให้; **@Column(nullable=false):** ไม่อนุญาต NULL ในข้อมูลหลัก

@OneToOne + @JoinColumn: Product ถือ detail_id และกำหนด unique=true เพื่อบังคับ 1:1 ส่วน cascade=ALL ส่งต่อ persist, merge และ remove ไปยังข้อมูลลูก

@OneToMany(mappedBy="product"): เชื่อมกับ Review.product ซึ่งเป็นผู้เก็บ FK จริง ใช้ orphanRemoval=true ให้ลบรีวิวเมื่อถอดออกจาก collection และ @OrderBy ให้เรียงรีวิวตาม id

```
public void addReview(Review review) {
    reviews.add(review);
    review.setProduct(this);
}
```

helper addReview() ทำให้ข้อมูลทั้ง Product.reviews และ Review.product สอดคล้องกันก่อนบันทึก ส่วน setDetail() ก็ตั้ง ProductDetail.product กลับมาที่สินค้าเช่นเดียวกัน

ProductDetail และ Review

```
package com.example.demo.model;
import jakarta.persistence.*;
@Entity
@Table(name = "product_details")
public class ProductDetail {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(length = 4000)
    private String description;
    @Column
    private String warranty;
    @Column
    private Double weight;
    @Column
    private String dimensions;
    @Column
    private String manufacturedCountry;
    @OneToOne(mappedBy = "detail")
    private Product product;
}
```

ProductDetail.product ใช้ mappedBy="detail" จึงเป็น inverse side ไม่สร้าง FK ชำในตาราง product_details ข้อมูลคำอธิบายกำหนดความยาวสูงสุด 4,000 ตัวอักษร

```
package com.example.demo.model;
import jakarta.persistence.*;
import java.time.LocalDate;
import org.hibernate.annotations.Check;
@Entity
@Table(name = "reviews")
@Check(constraints = "rating between 1 and 5")
public class Review {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(nullable = false)
    private String reviewer;
    @Column(nullable = false)
    private Integer rating;
    @Column(length = 2000)
    private String comment;
    @Column(nullable = false)
    private LocalDate reviewDate;
    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "product_id", nullable = false)
    private Product product;
}
```

Review.product ใช้ @ManyToOne และ @JoinColumn(name="product_id") จึงเป็นฝั่งที่เก็บ Foreign Key และไม่อนุญาต NULL การใช้ LAZY ช่วยลดการโหลดสินค้าเมื่ออ่านรีวิวแยก ส่วน @Check บังคับคะแนน 1-5 ที่ระดับฐานข้อมูล และ reviewDate ถูกกำหนดโดย Service

ธุรกรรมและการรักษาความสัมพันธ์

```

@Service
@Transactional
public class ProductService {
    private final ProductRepository products;
    private final DiscountContext discounts;
    public ProductService(ProductRepository products, DiscountContext discounts) {
        this.products = products;
        this.discounts = discounts;
    }
    @Transactional(readOnly = true)
    public List<Product> findAll() {
        List<Product> result = products.findAll();
        result.forEach(p -> p.setDiscountedPrice(discounts.calculate(p.getDiscountType(),
        p.getPrice())));
        return result;
    }
    @Transactional(readOnly = true)
    public Product findById(Long id) {
        return products.findById(id).orElseThrow(() -> new NoSuchElementException("ไม่พบสินค้า"));
    }
    public Product create(ProductForm form) {
        Product product = new Product();
        product.setDetail(new ProductDetail());
        copy(form, product);
        for (ReviewForm r : form.getReviews()) {
            if (r.hasContent()) product.addReview(toReview(r));
        }
        return products.save(product);
    }
    public Product update(Long id, ProductForm form) {
        Product product = findById(id);
        copy(form, product); // Update the existing detail; preserve its ID and all reviews.
        return products.save(product);
    }
    public void delete(Long id) { products.delete(findById(id)); }
    public void addReview(Long id, ReviewForm form) {
        Product product = findById(id);
        product.addReview(toReview(form));
        products.save(product);
    }
    public void deleteReview(Long id, Long reviewId) {
        Product product = findById(id);
        Review review = product.getReviews().stream().filter(r -> r.getId().equals(reviewId))
        .findFirst().orElseThrow(() -> new NoSuchElementException("ไม่พบรีวิวของสินค้า"));
        product.removeReview(review);
    }
}

```

Constructor Injection: Spring ส่ง ProductRepository และ DiscountContext เข้า constructor แล้วเก็บใน final fields ทำให้ dependency ชัดเจนและสามารถทดแทนด้วยตัวทดสอบได้

update() โหลด Entity เดิมก่อน copy เฉพาะข้อมูลที่แก้ไขได้ จึงรักษา detail.id และรีวิวดเดิม ส่วน delete() ใช้ JPA cascade ลบข้อมูลลูกทั้งหมดภายใน transaction เดียว การลบรีวิวจะตรวจด้วยว่ารีวิวนั้นเป็นของสินค้าที่ระบุจริง

รับข้อมูล ตรวจสอบ และส่งผลกลับ

```
@Controller
public class ProductController {
    private final ProductService service;
    private final DiscountContext discounts;
    public ProductController(ProductService service, DiscountContext discounts) {
        this.service = service; this.discounts = discounts;
    }
    @InitBinder("product")
    void bindProduct(WebDataBinder binder) {
        binder.setAutoGrowCollectionLimit(1);
        binder.setAllowedFields("name", "category", "brand", "stock", "price", "discountType",
            "detail.description", "detail.warranty", "detail.weight", "detail.dimensions",
            "detail.manufacturedCountry", "reviews[0].reviewer", "reviews[0].rating",
            "reviews[0].comment");
    }
    @GetMapping("/") public String home() { return "redirect:/products"; }
    @GetMapping("/products")
    public String list(Model model) { model.addAttribute("products", service.findAll()); return
        "products/list"; }
    @GetMapping("/products/add")
    public String add(Model model) { model.addAttribute("product", new ProductForm()); return
        "products/add"; }
    @PostMapping("/products/save")
    public String save(@Valid @ModelAttribute("product") ProductForm form, BindingResult errors,
        RedirectAttributes flash) {
        validateDiscount(form, errors);
        if (errors.hasErrors()) return "products/add";
        service.create(form); flash.addFlashAttribute("message", "เพิ่มสินค้าเรียบร้อยแล้ว");
        return "redirect:/products";
    }
    @GetMapping("/products/edit/{id}")
    public String edit(@PathVariable Long id, Model model) {
        model.addAttribute("product", service.editForm(id)); return "products/edit";
    }
    @PostMapping("/products/update/{id}")
    public String update(@PathVariable Long id, @Valid @ModelAttribute("product") ProductForm form,
        BindingResult errors, RedirectAttributes flash) {
        service.findById(id); form.setId(id); validateDiscount(form, errors);
        if (errors.hasErrors()) return "products/edit";
        service.update(id, form); flash.addFlashAttribute("message", "แก้ไขสินค้าเรียบร้อยแล้ว");
        return "redirect:/products";
    }
    @GetMapping("/products/delete/{id}")
    public String confirmDelete(@PathVariable Long id, Model model) {
        model.addAttribute("product", service.findById(id)); return "products/delete";
    }
    @PostMapping("/products/delete/{id}")
    public String delete(@PathVariable Long id, RedirectAttributes flash) {
        service.delete(id); flash.addFlashAttribute("message", "ลบสินค้า ข้อมูลเสริม
        และรีวิวเรียบร้อยแล้ว");
        return "redirect:/products";
    }
}
```

Controller รับ ProductService และ DiscountContext ผ่าน constructor ไม่สร้าง dependency เอง ฟอรั่มใช้ DTO เพื่อไม่เปิดให้แก้ id หรือ FK โดยตรง @InitBinder จำกัดเฉพาะช่องของเอนทิตี ส่วน @Valid และ BindingResult ส่งข้อผิดพลาดกลับไปยังหน้าฟอรั่มเดิม

โค้ดเต็มของ Service, Controller, helper methods, ฟอรั่ม และส่วนจัดการรีวิวยู่ใน src/main/java/com/example/demo ของโปรเจกต์

ผลทดสอบและการใช้งาน

ผลอัตโนมัติ: ผ่าน 6 tests, 0 failures, 0 errors

1. CRUD ครบวงจร: สร้างสินค้าและข้อมูลลูก เพิ่มรีวิวที่สอง แก้ไขแล้ว detail.id และรีวิวยังอยู่ ลบรีวิวและลบสินค้าพร้อมข้อมูลลูก
2. ไม่สร้างแถวรีวิวหากเว้นรีวิวแรกว่าง
3. ปฏิเสธข้อมูลสินค้าไม่ครบและรีวิวที่มีความเห็นแต่ไม่มีชื่อ
4. ปฏิเสธคะแนนนอกช่วง 1-5 และคืน 404 เมื่อไม่พบสินค้า
5. ป้องกันการลบรีวิวโดยใช้ id ของสินค้าอื่น
6. ตรวจสอบ NONE, MEMBER, SEASONAL การปิดทศนิยม และชนิดส่วนลดที่ไม่รองรับ

ชุดทดสอบ SpringBootTest + MockMvc ใช้ H2 ใน PostgreSQL mode เพื่อให้รันได้โดยไม่ต้องมี PostgreSQL ส่วนภาพหน้าจอและการตรวจ cascade เพิ่มเติมใช้แอปกับ PostgreSQL 18.4 จริง จึงแยกหลักฐานทั้งสองแบบอย่างชัดเจน

สภาพแวดล้อมทดสอบมีปัญหา AccessDeniedException ของ javac จึงคอมไพล์ test ด้วย Eclipse ECJ แล้วรัน Maven surefire:test ซึ่งผ่านครบ 6 กรณี ตัวแอป package สำเร็จด้วย Maven และเปิดบน Java 21; โปรเจกต์กำหนด Java release 17

วิธีรันบนเครื่องผู้ใช้ (PowerShell)

```
psql -U postgres -c "CREATE DATABASE lab8shop;"
$env:DB_PASSWORD="your-postgres-password"
.\mvnw.cmd spring-boot:run
# Open http://localhost:8080/products
# Tests: .\mvnw.cmd test
```

ตั้ง DB_URL, DB_USERNAME และ PORT เพิ่มได้ตามต้องการ README.md อธิบายวิธีรันและการนำขึ้น GitHub ชื่อ lab8-673380062-2-sec1 โดยรายงานนี้ยังไม่ระบุ URL repository ของนักศึกษา เพราะยังไม่ได้สร้างหรืออัปโหลดเข้า GitHub

Create - เพิ่มสินค้าและข้อมูลลูกค้า

กรอกสินค้า iPhone 15 Pro (673380062-2 SEC 1), ราคา 40,000 บาท ส่วนลด MEMBER พร้อม ProductDetail และรีวิวกแรก

+ เพิ่มสินค้าใหม่

ข้อมูลสินค้า (Product)

ชื่อสินค้า

iPhone 15 Pro (673380062-2 SEC 1)

ยี่ห้อ

Apple

ราคาปกติ (บาท)

40000

หมวดหมู่

Electronics

จำนวนในคลัง

10

ประเภทส่วนลด

ส่วนลดสมาชิก (10%)

ข้อมูลเพิ่มเติมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1

รายละเอียดสินค้า

สมาร์ตโฟนจอ 6.1 นิ้ว กล้อง 48 MP ความจุ 256 GB

ระยะเวลาประกัน

1 Year

น้ำหนัก (กิโลกรัม)

0.19

ขนาด (กว้าง x ยาว x สูง)

ประเทศที่ผลิต

ส่วนบน: ข้อมูลสินค้าและรายละเอียดเพิ่มเติม

รายละเอียดสินค้า

สมาร์ตโฟนจอ 6.1 นิ้ว กล้อง 48 MP ความจุ 256 GB

ระยะเวลาประกัน

1 Year

น้ำหนัก (กิโลกรัม)

0.19

ขนาด (กว้าง x ยาว x สูง)

15x7x0.8 cm

ประเทศที่ผลิต

China

★ รีวิวสินค้า (Review) — ความสัมพันธ์ 1:N

เห็นรีวิวแรกพร้อมกับสินค้า (สามารถเห็นรีวิวเพิ่มเติมภายหลังได้)

ชื่อผู้รีวิว

Student 673380062-2

คะแนน (1-5)

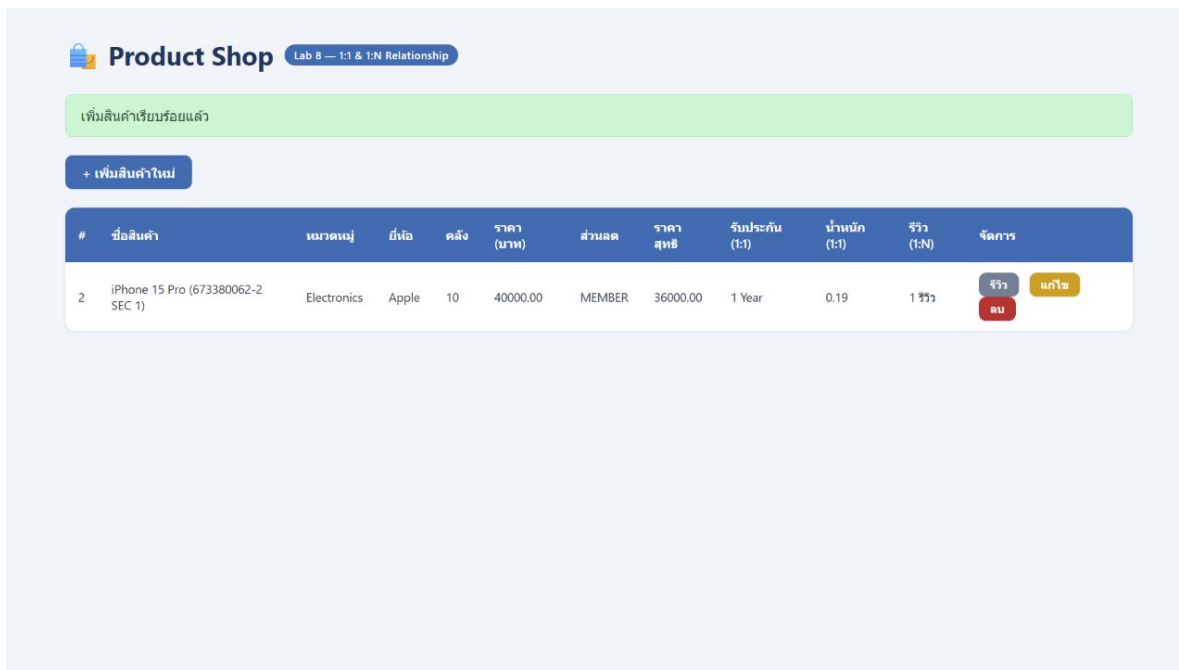
★★★★★ (5)

ความเห็น

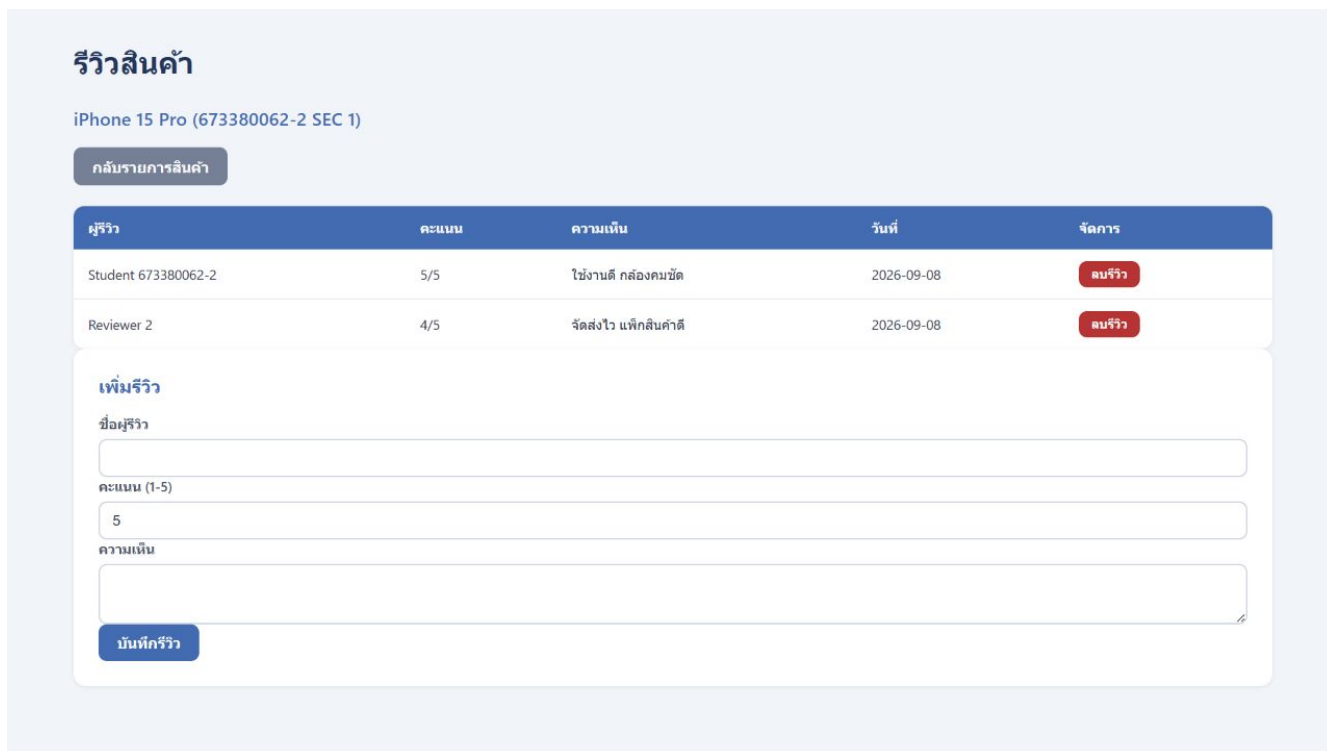
ใช้งานดี กล้องคมชัด

ส่วนล่าง: ข้อมูลเสริมที่เหลือและรีวิวก่อนบันทึก

Read และรีวิวแบบ 1:N



หลังสร้างสินค้า: ราคา 40,000 บาท ลดสมาชิก 10% เหลือ 36,000 บาท และมีรีวิวหนึ่งรายการ



เพิ่มรีวิวที่สองให้สินค้ารายการเดิมสำเร็จ รีวิวทั้งสองอ้างถึง product_id เดียวกัน และมีวันที่รีวิวที่กำหนดจากระบบ

Update - แก้ไขสินค้า

เปลี่ยนคลังเป็น 8, ราคาเป็น 39,000 บาท, ส่วนลด SEASONAL และรับประกัน 2 Years โดยแก้ไขข้อมูล ProductDetail รายการเดิม

 **แก้ไขสินค้า**

 **ข้อมูลสินค้า (Product)**

ชื่อสินค้า

iPhone 15 Pro (673380062-2 SEC 1)

หมวดหมู่

Electronics

ยี่ห้อ

Apple

จำนวนในคลัง

8

ราคาปกติ (บาท)

39000

ประเภทส่วนลด

ส่วนลดเทศกาล (20%)

 **ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1**

รายละเอียดสินค้า

สมาร์ทโฟนจอ 6.1 นิ้ว กล้อง 48 MP ความจุ 256 GB

ระยะเวลาประกัน

2 Years

น้ำหนัก (กิโลกรัม)

0.19

ขนาด

ประเทศที่ผลิต

ส่วนบนของฟอร์มแก้ไข

ชื่อสินค้า

iPhone 15 Pro (673380062-2 SEC 1)

หมวดหมู่

Electronics

ยี่ห้อ

Apple

จำนวนในคลัง

8

ราคาปกติ (บาท)

39000

ประเภทส่วนลด

ส่วนลดเทศกาล (20%)

 **ข้อมูลเสริมสินค้า (ProductDetail) — ความสัมพันธ์ 1:1**

รายละเอียดสินค้า

สมาร์ทโฟนจอ 6.1 นิ้ว กล้อง 48 MP ความจุ 256 GB

ระยะเวลาประกัน

2 Years

น้ำหนัก (กิโลกรัม)

0.19

ขนาด

15x7x0.8 cm

ประเทศที่ผลิต

China

 **บันทึกการแก้ไข**

ยกเลิก

ส่วนล่างแสดงข้อมูลเสริมและปุ่มบันทึกการแก้ไข

ผลหลังแก้ไข

 **Product Shop**

Lab 8 — 1:1 & 1:N Relationship

แก้ไขสินค้าเรียบร้อยแล้ว

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
2	iPhone 15 Pro (673380062-2 SEC 1)	Electronics	Apple	8	39000.00	SEASONAL	31200.00	2 Years	0.19	2 รีวิว	รีวิว ลบ แก้ไข

ราคาสุทธิเปลี่ยนเป็น 31,200 บาท ($39,000 \times 0.80$) คลังเหลือ 8 และแสดงรับประกัน 2 Years โดยจำนวนรีวิวยังเป็น 2 รีวิว

สิ่งที่ตรวจสอบเพิ่มเติม

การแก้ไขไม่เปลี่ยน detail.id และไม่สร้าง ProductDetail ใหม่ ชุดทดสอบตรวจจำนวนแถวและ id ก่อน-หลังอัปเดต รวมทั้งตรวจว่าไม่มีรีวิวถูกลบโดยไม่ตั้งใจ

ระบบไม่รับ id จากฟอร์มมาเขียนทับ Entity โดยตรง แต่ใช้ id จากเส้นทาง URL เพื่อค้นหา Entity เดิมก่อนอัปเดต

Delete - ก่อนและหลังลบ


ยืนยันการลบสินค้า

คุณต้องการลบสินค้านี้ใช่ไหม? การดำเนินการนี้ไม่สามารถย้อนกลับได้

ชื่อสินค้า:	iPhone 15 Pro (673380062-2 SEC 1)
หมวดหมู่:	Electronics
ยี่ห้อ:	Apple
ราคา:	39000.00 บาท
รายละเอียด:	สมาร์ทโฟนจอ 6.1 นิ้ว กล้อง 48 MP ความจุ 256 GB
รับประกัน:	2 Years

⚠ ข้อมูลเสริม (ProductDetail) จะถูกลบพร้อมกันด้วย เนื่องจาก CascadeType.ALL

ยืนยันลบ

ยกเลิก

หน้ายืนยันลบแสดงข้อมูลสินค้าและข้อมูลเสริมให้ตรวจสอบก่อนส่ง POST

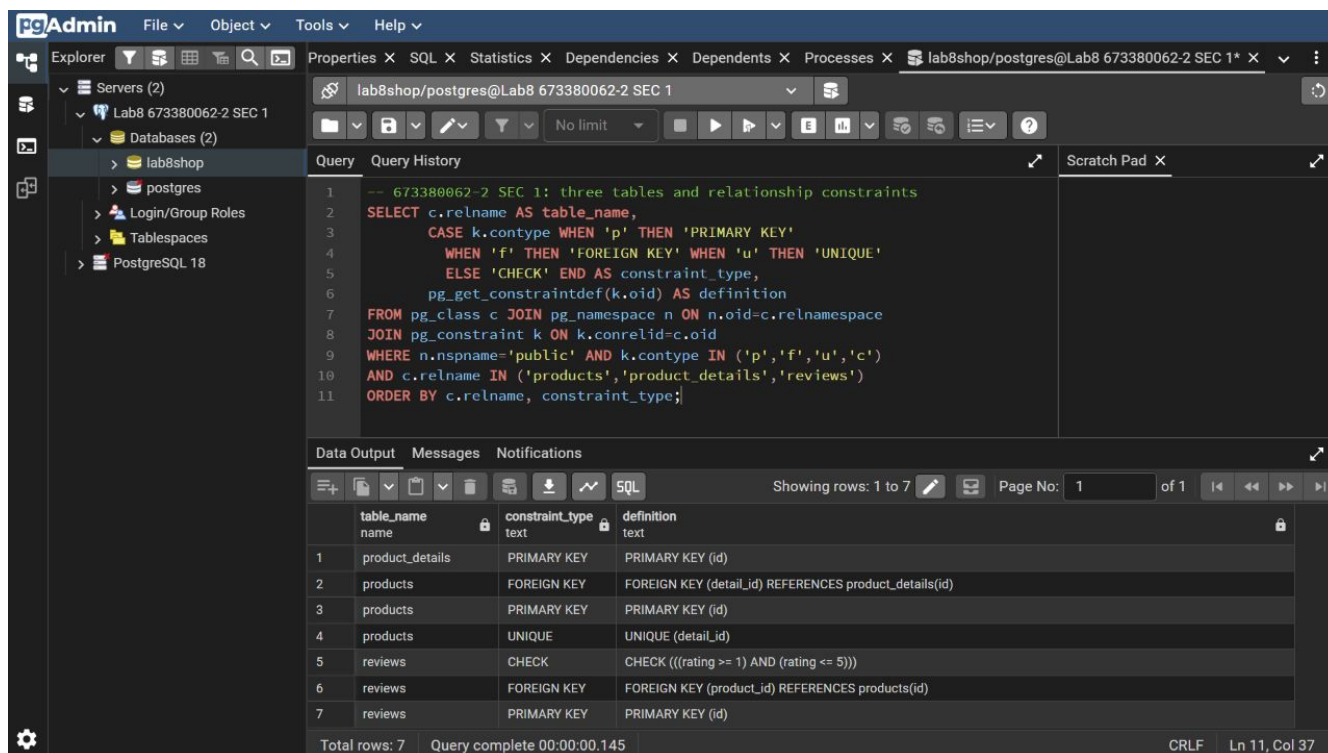

Product Shop
Lab 8 — 1:1 & 1:N Relationship

+ เพิ่มสินค้าใหม่

#	ชื่อสินค้า	หมวดหมู่	ยี่ห้อ	คลัง	ราคา (บาท)	ส่วนลด	ราคาสุทธิ	รับประกัน (1:1)	น้ำหนัก (1:1)	รีวิว (1:N)	จัดการ
ยังไม่มีสินค้า — กด "+ เพิ่มสินค้าใหม่" เพื่อเริ่มต้น											

หลังส่งค่าขอลบ รายการสินค้าว่าง ตรวจสอบ PostgreSQL เพิ่มแล้วพบ products=0, product_details=0 และ reviews=0
ยืนยันว่าลบข้อมูลลูกครบ ไม่มี orphan record

Database - ตารางและ Foreign Key



The screenshot shows the pgAdmin interface with a SQL query executed. The query is designed to list constraints for three specific tables: product_details, products, and reviews. The results are displayed in a table with 7 rows.

```

1  -- 673380062-2 SEC 1: three tables and relationship constraints
2  SELECT c.relname AS table_name,
3         CASE k.contype WHEN 'p' THEN 'PRIMARY KEY'
4              WHEN 'f' THEN 'FOREIGN KEY' WHEN 'u' THEN 'UNIQUE'
5              ELSE 'CHECK' END AS constraint_type,
6         pg_get_constraintdef(k.oid) AS definition
7  FROM pg_class c JOIN pg_namespace n ON n.oid=c.relnamespace
8  JOIN pg_constraint k ON k.conrelid=c.oid
9  WHERE n.nspname='public' AND k.contype IN ('p','f','u','c')
10 AND c.relname IN ('products','product_details','reviews')
11 ORDER BY c.relname, constraint_type;

```

	table_name	constraint_type	definition
1	product_details	PRIMARY KEY	PRIMARY KEY (id)
2	products	FOREIGN KEY	FOREIGN KEY (detail_id) REFERENCES product_details(id)
3	products	PRIMARY KEY	PRIMARY KEY (id)
4	products	UNIQUE	UNIQUE (detail_id)
5	reviews	CHECK	CHECK (((rating >= 1) AND (rating <= 5)))
6	reviews	FOREIGN KEY	FOREIGN KEY (product_id) REFERENCES products(id)
7	reviews	PRIMARY KEY	PRIMARY KEY (id)

Total rows: 7 Query complete 00:00:00.145 CRLF Ln 11, Col 37

ภาพจาก pgAdmin ที่เชื่อมต่อ PostgreSQL ของโปรเจกต์ แสดง constraints ของทั้งสามตารางจาก system catalog จริง

products: PRIMARY KEY (id), FOREIGN KEY (detail_id) REFERENCES product_details(id) และ UNIQUE (detail_id) บังคับความสัมพันธ์ 1:1

reviews: PRIMARY KEY (id), FOREIGN KEY (product_id) REFERENCES products(id) และ CHECK คะแนน 1-5 รองรับความสัมพันธ์ 1:N

product_details: PRIMARY KEY (id) ส่วนการลบข้อมูลลูกเกิดจาก JPA cascade ใน Service ไม่ใช่การตั้ง ON DELETE CASCADE ที่ฐานข้อมูล

แหล่งโจทย์และไฟล์ประกอบ

GitHub: [PANANG6425/LAB08_Table_Relationships](https://github.com/PANANG6425/LAB08_Table_Relationships)

ASSIGNMENT.md เก็บโจทย์ต้นฉบับ; docs/screenshots เก็บภาพจริงทั้งหมด; docs/database-snapshot.sql เป็น snapshot ก่อนลบสำหรับศึกษาตารางและข้อมูลตัวอย่าง; docs/test-results.txt เก็บผลทดสอบ